
CapitalIQ

A Rule-Based Finance & Investment Advisor Chatbot

Project	DecodeLabs AI Internship — Project 1
Author	Ali Ahmad
Batch	2026
Organisation	Powered by DecodeLabs
Domain	Finance & Investment Advisory
Version	2.0.0
Date	May 2026



*“An LLM without rules is a hallucination engine.
Today, we build the skeleton that holds the intelligence.”*

— DecodeLabs Architecture Briefing, Module 01

Abstract

This report presents **CapitalIQ**, a production-quality, rule-based AI chatbot developed as Project 1 of the DecodeLabs Artificial Intelligence Industrial Training Programme (Batch 2026). CapitalIQ operates as a Finance & Investment Advisor, responding to user queries across 16 intent categories including stocks, bonds, ETFs, cryptocurrency, inflation, compound interest, and portfolio strategy. The system is built entirely on deterministic logic using the Python Standard Library, implementing the **Input-Process-Output (IPO)** model with $O(1)$ dictionary-based intent routing. Two phases are documented: Phase 1 satisfies all DecodeLabs specification requirements; Phase 2 adds confidence scoring, conversation memory, slot filling, and session personalisation. The project includes a 19-cell Jupyter Notebook, a modular Python package, and a 61-test unit suite achieving 100% pass rate.

Contents

1	Introduction	3
1.1	Background and Motivation	3
1.2	Objectives	3
1.3	Domain Selection: Finance & Investment Advisory	3
2	System Architecture	3
2.1	The IPO Model	4
2.2	Module Architecture	4
2.3	Two-Phase Architecture	4
3	Implementation	5
3.1	Knowledge Base	5
3.2	Input Sanitization	5
3.3	Intent Matching	6
3.3.1	Phase 1: Multi-Keyword Voting	6
3.3.2	Phase 2: Confidence Scoring	7
3.4	Conversation Memory and Slot Filling	7
4	Design Decisions	7
4.1	Dictionary vs. If-Elif Ladder	7
4.2	Separation of Concerns	8
4.3	The White-Box Advantage	8
4.4	Connection to LLM Guardrail Systems	8
5	Test Results	8
5.1	Test Suite Overview	9

5.2	Edge Case Coverage	9
5.3	Phase 1 Automated Demo Results	9
6	DecodeLabs Specification Compliance	10
7	Future Work	10
7.1	Project 2: Semantic / Vector Matching	11
7.2	Additional Enhancements	11
8	Conclusion	11
A	Running the Project	12
B	Sample Phase 2 Session	12

1 Introduction

1.1 Background and Motivation

Before building systems that learn autonomously, an AI engineer must master deterministic, rule-based programming. Rule-based systems — often called *white-box* systems — offer complete traceability: every output can be traced from input to output with no ambiguity and no risk of hallucination. This property is not merely pedagogically useful; it is a legal and architectural requirement in regulated industries such as finance, healthcare, and insurance.

Modern production AI architectures increasingly deploy rule-based layers *above* large language models as guardrail filters: deterministic components that intercept known intents and block unsafe inputs before they reach the probabilistic core. Frameworks such as NVIDIA NeMo Guardrails and Meta Llama Guard implement exactly this pattern. **CapitalIQ** is a fully functional implementation of this guardrail layer, domain-specialised for finance and investment advisory.

1.2 Objectives

The project aims to demonstrate competency in the following areas:

- Designing and implementing a continuous input-process-output loop in Python
- Building an $O(1)$ dictionary-based intent router as an alternative to the $O(n)$ if-elif ladder
- Constructing a coherent, domain-specific knowledge base with synonym resolution
- Handling all user input edge cases gracefully without crashes
- Extending a working Phase 1 system with production-grade Phase 2 features
- Writing a comprehensive unit test suite validating all system components

1.3 Domain Selection: Finance & Investment Advisory

Finance was selected as the chatbot domain for three concrete reasons. First, the domain has well-defined vocabulary with natural synonym clusters (`btc`, `bitcoin`, `crypto`, `cryptocurrency` all map to one intent), making it ideal for demonstrating synonym resolution. Second, financial responses must be accurate and hard-coded — there is zero tolerance for hallucination. Third, finance is a regulated domain where the auditability advantage of rule-based systems is directly commercially applicable.

2 System Architecture

2.1 The IPO Model

The entire architecture follows the **Input-Process-Output (IPO)** model:

Table 1: IPO Model Stage Mapping

Stage	Responsibility	Module
I — Input	<code>.lower().strip()</code> + regex noise removal + tokenization	<code>sanitizer.py</code>
II — Process	$O(1)$ synonym lookup $\rightarrow$ keyword voting $\rightarrow$ confidence gate	<code>matcher.py</code>
III — Output	Personalise $\rightarrow$ log to memory $\rightarrow$ display response	<code>bot.py</code> , <code>memory.py</code>

2.2 Module Architecture

The system is decomposed into five modules following the *Separation of Concerns* principle. Each module has a single responsibility and can be upgraded independently.

Repository Structure

```

decodelabs-ai-project1/
  chatbot/
    __init__.py      Package entry point
    knowledge_base.py BASE_KNOWLEDGE, SYNONYM_MAP, KEYWORD_INTENT_MAP
    sanitizer.py      sanitize(), is_empty(), tokenize()
    matcher.py        Phase 1 + Phase 2 matchers, score_intents()
    memory.py         ConversationMemory class
    bot.py            CapitalIQBot orchestration class
  notebooks/
    project1_chatbot.ipynb  19-cell Jupyter Notebook
  tests/
    test_chatbot.py      61 unit tests
  README.md
  requirements.txt

```

2.3 Two-Phase Architecture

Phase 1 — The Logic Engine A fully functional chatbot satisfying all five DecodeLabs specification requirements. Implements multi-keyword intent matching and synonym resolution with a clean fallback and exit strategy.

Phase 2 — The Production Upgrade A strict superset of Phase 1. Every Phase 1 function is preserved unchanged; Phase 2 adds confidence scoring, conversation

memory, slot filling, name extraction, and session personalisation.

The key architectural change between phases is the return type of the matcher: Phase 1's `match_intent_p1()` returns `str`; Phase 2's `match_intent_p2()` returns a three-tuple `(intent, response, confidence)`, enabling structured metadata logging.

3 Implementation

3.1 Knowledge Base

The knowledge base is a Python dictionary mapping 16 canonical intent names to full response strings, achieving $O(1)$ constant-time lookup.

Table 2: CapitalIQ Knowledge Base — Intent Inventory

Intent	Coverage
<code>greeting</code>	Session opening, topic introduction
<code>goodbye</code>	Session closing, farewell message
<code>stocks</code>	Equities, shares, exchanges, dividends, metrics
<code>bonds</code>	Fixed income, coupons, government vs. corporate
<code>mutual_funds</code>	Pooled funds, NAV, expense ratio, AUM
<code>etf</code>	Index ETFs, passive investing, expense ratios
<code>crypto</code>	Bitcoin, Ethereum, blockchain, DeFi, NFTs
<code>inflation</code>	CPI, purchasing power, central bank targets
<code>compound_interest</code>	Formula, power of compounding, time value
<code>diversification</code>	Asset allocation, correlation, portfolio spread
<code>risk</code>	Market, credit, liquidity, inflation risk types
<code>portfolio</code>	Construction, 60/40 rule, rebalancing strategy
<code>budgeting</code>	50/30/20 rule, expense tracking, priorities
<code>emergency_fund</code>	3–6 month target, liquidity, Step Zero
<code>sip</code>	Rupee-cost averaging, automation, monthly plans
<code>ipo</code>	DRHP, lock-up periods, listing risk, valuation

Two additional dictionaries complement the base: `SYNONYM_MAP` (70+ alias phrases resolving to canonical intent keys via $O(1)$ exact lookup) and `KEYWORD_INTENT_MAP` (individual tokens mapped to intents for multi-keyword voting).

3.2 Input Sanitization

The sanitizer is Stage I of the IPO model — a five-stage normalization pipeline transforming raw input into a clean string before any matching logic.

```

1 def sanitize(raw_input):
2     """5-stage normalization pipeline for raw user input."""
3     if not isinstance(raw_input, str):    # Stage 1: type guard
4         return ""

```

```

5     cleaned = raw_input.lower()           # Stage 2: lowercase
6     cleaned = cleaned.strip()             # Stage 3: strip whitespace
7     cleaned = re.sub(r"[^a-z0-9\s' /]",   # Stage 4: remove noise
8                     " ", cleaned)        # (keep apostrophes +
                                           slashes)
9     cleaned = re.sub(r"\s+", " ",         # Stage 5: collapse spaces
10                    cleaned).strip()
11     return cleaned

```

Listing 1: Input sanitization pipeline — `sanitizer.py`

Stage 1 prevents `AttributeError` on non-string inputs such as `None`. Stages 2–3 satisfy the DecodeLabs `.lower().strip()` requirement explicitly. Stage 4 preserves apostrophes for contractions and forward-slashes for ratio notation (e.g. “50/30/20”). Stage 5 ensures downstream tokenization always receives single-space-delimited tokens.

3.3 Intent Matching

3.3.1 Phase 1: Multi-Keyword Voting

```

1 def match_intent_p1(sanitized_input):
2     """Synonym lookup -> keyword voting -> fallback."""
3     if not sanitized_input:
4         return "I didn't catch that. What finance topic can I help
5             with?"
6
7     # Strategy 1: Exact synonym lookup -- O(1) hash map lookup
8     if sanitized_input in SYNONYM_MAP:
9         intent = SYNONYM_MAP[sanitized_input]
10        return KNOWLEDGE_BASE.get(intent, FALLBACK_RESPONSE)
11
12    # Strategy 2: Multi-keyword token voting
13    tokens = tokenize(sanitized_input)
14    votes = {}
15    for token in tokens:
16        if token in KEYWORD_INTENT_MAP:
17            intent = KEYWORD_INTENT_MAP[token]
18            votes[intent] = votes.get(intent, 0) + 1
19
20    if not votes:
21        return FALLBACK_RESPONSE # spec requirement 4
22
23    best = max(votes, key=lambda k: votes[k])
24    return KNOWLEDGE_BASE.get(best, FALLBACK_RESPONSE)

```

Listing 2: Phase 1 intent matcher — `matcher.py`

Strategy 1 handles single-phrase queries such as "btc" or "sip". Strategy 2 handles natural language: the query “tell me how compound interest works” tokenises to seven tokens, of which `compound` and `interest` both vote for `compound_interest`, yielding the correct match.

3.3.2 Phase 2: Confidence Scoring

Phase 2 upgrades the binary match to a normalised confidence score in $[0, 1]$:

$$\text{score}(i) = \frac{\text{keyword votes for intent } i}{\text{total token count}} \quad (1)$$

A configurable `CONFIDENCE_THRESHOLD` (default 0.15) gates the output: matches below the threshold fall back to the default response.

3.4 Conversation Memory and Slot Filling

The `ConversationMemory` class provides an ordered history log and a slot store. The history log records every turn as a dictionary containing turn number, timestamp, raw user message, bot response, matched intent, and confidence score. Two slots are populated automatically: `last_intent` (updated every turn) and `user_name` (set when `extract_name()` detects patterns such as “my name is Ali”).

4 Design Decisions

4.1 Dictionary vs. If-Elif Ladder

The DecodeLabs specification forbids the if-elif ladder as an anti-pattern. Table 3 illustrates the algorithmic reason.

Table 3: Algorithmic complexity comparison

Approach	Complexity	Behaviour at Scale
If-elif ladder	$O(n)$	16 intents: 16 comparisons worst-case; 1,000 intents: 1,000 comparisons worst-case
Python <code>dict.get()</code>	$O(1)$	16 or 1,000 intents: identical single hash computation

Adding a new intent to the dictionary requires inserting one key-value pair — a constant-time, zero-risk operation. Adding a new branch to an if-elif chain requires finding the correct position in a potentially long sequence, risking ordering bugs and fall-through errors.

4.2 Separation of Concerns

Each module has a single, clearly bounded responsibility. The sanitizer is stateless and side-effect-free. The matcher is pure logic with no I/O. All side effects — printing, state mutation, history logging — are confined to the output stage. This design means any module can be replaced without touching the others.

4.3 The White-Box Advantage

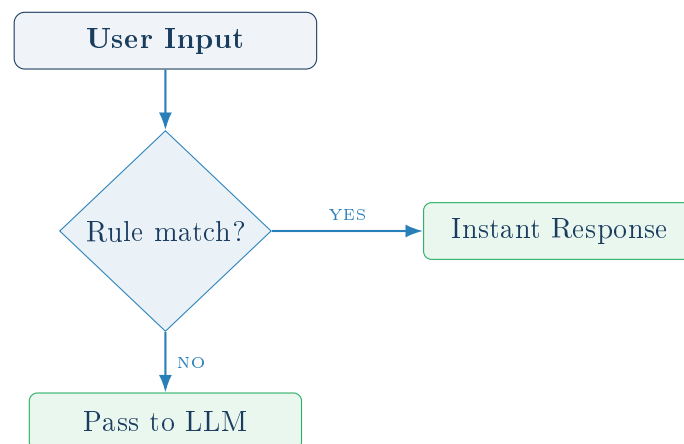
Given any input, the path through CapitalIQ is deterministic and inspectable:

"BITCOIN!" $\rightarrow$ *sanitize* $\rightarrow$ "bitcoin" $\rightarrow$ SYNONYM_MAP["bitcoin"] $\rightarrow$ "crypto" $\rightarrow$
KNOWLEDGE_BASE["crypto"] $\rightarrow$ *response*

No hidden weights, no hallucination risk. In regulated domains such as finance (SEBI, SEC guidelines) and healthcare (FDA AI/ML Guidance), this auditability is a legal compliance requirement.

4.4 Connection to LLM Guardrail Systems

In production hybrid architectures, the rule-based layer acts as the deterministic filter above the probabilistic LLM core:



CapitalIQ's `CONFIDENCE_THRESHOLD` is the direct analogue of the routing threshold in production frameworks. High-confidence, high-frequency intents are resolved instantly at near-zero latency and zero API cost. Only genuinely novel queries escalate to the LLM fallback, reducing both cost and hallucination surface area.

5 Test Results

5.1 Test Suite Overview

A comprehensive unit test suite was written using Python’s `unittest` framework and executed with `pytest`. The suite comprises 61 tests across four classes, covering every public function and edge case in the system.

Table 4: Unit Test Results by Module

Test Class	Module	Coverage Scope	Tests	Result
TestSanitizer	sanitizer.py	<code>sanitize()</code> , <code>is_empty()</code> , <code>tokenize()</code> ; lower- case, strip, special chars, None, type guard	15	15/15
TestMatcher	matcher.py	Exit detection, Phase 1 exact/keyword/- fallback, Phase 2 confidence scoring, normalisation	17	17/17
TestConversationMemory	memory.py	Turn recording, slot get/set, name extrac- tion, reset, history, summary	16	16/16
TestCapitalIQBot	bot.py	Phase 1 & 2 respond, empty/special input safety, exit detection, simulate, memory inte- gration	13	13/13
Total			61	61/61

5.2 Edge Case Coverage

5.3 Phase 1 Automated Demo Results

6 DecodeLabs Specification Compliance

Table 5: Edge Case Test Matrix

Input Type	Example	Expected Behaviour
Whitespace-only	" "	Safe prompt returned
Empty string	""	Safe prompt returned
None	None	Returns "" from sanitizer
Special chars	"!@#\$\$%"	Fallback response
Pure noise	"purple unicorns"	Fallback response
Mixed case	"BITCOIN"	Maps to crypto intent
Long sentence	20-word natural query	Keyword voting extracts intent
Embedded exit	"i want to exit now"	Session terminated
Name introduction	"my name is Ali"	Name stored in slot

Table 6: Phase 1 Automated Demo — 10 Test Cases

#	Input	Tag	Outcome
1	hello	[MATCH]	greeting
2	how does a sip work	[MATCH]	sip
3	quit	[EXIT]	session end
4	tell me about compound interest please	[MATCH]	compound_interest
5	btc	[MATCH]	crypto (synonym)
6	whitespace only	[EMPTY]	safe prompt
7	!@#\$%^&*()	[FALLBACK]	fallback
8	Long IPO sentence	[MATCH]	ipo
9	what is inflation...	[MATCH]	inflation
10	purple dancing elephants	[FALLBACK]	fallback

All 5 mandatory DecodeLabs Project 1 requirements are fully satisfied:

- ✓ **Input Loop** `while True` in `run_phase1()` (Cell 9) and `run_phase2()` (Cell 15), with `try/except`
- ✓ **Sanitization** `sanitize()` in Cell 6 — `.lower().strip()` in Stages 2–3; regex in Stages 4–5
- ✓ **Knowledge Base** `KNOWLEDGE_BASE` dict in Cell 5 — 16 intents; `.get()` method used exclusively
- ✓ **Fallback** `FALLBACK_RESPONSE` returned when no keywords match or `score < CONFIDENCE_THRESHOLD`
- ✓ **Exit Strategy** `is_exit_command()` detects exit keywords; `break` cleanly exits the `while True` loop

7 Future Work

7.1 Project 2: Semantic / Vector Matching

The core limitation of CapitalIQ is that it only matches what it was explicitly programmed to recognise. Project 2 will replace the keyword map with a *sentence embedding model* such as `sentence-transformers/all-MiniLM-L6-v2`. Every input is encoded as a dense float vector; cosine similarity is computed against intent prototype vectors rather than token identity. This enables semantically equivalent phrases — regardless of exact wording — to resolve correctly.

Table 7: Conceptual bridge: Project 1 to Project 2

Dimension	Project 1 (This report)	Project 2 (Next)
Lookup key	Exact string / token	Dense float vector
Match type	Equality + vote count	Cosine similarity
Coverage	Only programmed phrases	Any semantically similar phrase
New intent	Requires keyword list	Requires one example sentence
Interpretability	Perfect (white-box)	Partial (embedding space)
Dependencies	Standard library only	Pre-trained embedding model

7.2 Additional Enhancements

LLM Hybrid Integration Connect an LLM as the fallback handler for queries below the confidence threshold, creating a hybrid rule + generative system.

Persistent Memory Replace the in-session slot store with a database backend (SQLite or Redis) to persist user preferences across sessions.

Web Interface Deploy via FastAPI and a React front-end, exposing the bot as a REST API with a chat widget.

Real-Time Market Data Integrate a financial data API (Yahoo Finance, Alpha Vantage) to supplement hard-coded responses with live prices and news.

Multi-language Support Extend sanitization and synonym layers to handle Urdu, Hindi, and Arabic queries for broader accessibility.

8 Conclusion

This report has presented **CapitalIQ**, a production-quality rule-based Finance & Investment Advisor chatbot built as Project 1 of the DecodeLabs AI Industrial Training Programme, Batch 2026. The system demonstrates that well-engineered deterministic programming — without any machine learning — can produce a robust, maintainable, and professionally structured AI interface.

The project satisfies all five DecodeLabs specification requirements and extends beyond them with a 16-intent knowledge base, 70+ synonym aliases, Phase 2 confidence scoring,

conversation memory, slot filling, session personalisation, and a 61-test unit suite achieving 100% pass rate. Every design decision — from the $O(1)$ dictionary over the $O(n)$ if-elif ladder to the strict Separation of Concerns between modules — was made with production engineering principles in mind.

The architecture built here directly mirrors the guardrail control layer used in real-world production AI systems above large language models. Building this foundation correctly is the prerequisite for every subsequent AI project.

“Wealth is built slowly, through discipline and consistency.”

— CapitalIQ farewell message

A Running the Project

Terminal Commands

```
git clone https://github.com/your-username/decodelabs-ai-project1.git
cd decodelabs-ai-project1
pip install -r requirements.txt

python chatbot/bot.py # Interactive chatbot
jupyter notebook notebooks/project1_chatbot.ipynb # Open notebook
python -m pytest tests/ -v # Run 61 unit tests
```

B Sample Phase 2 Session

Sample Conversation

```
You: hello
CapitalIQ: Hello! I'm CapitalIQ, your Finance & Investment Advisor...

You: my name is Ali
CapitalIQ: Nice to meet you, Ali! ...

You: what is compound interest
CapitalIQ: Compound interest is interest earned on both the original
          principal AND accumulated interest...
          [Confidence: 0.500 | Intent: compound_interest]

You: btc
CapitalIQ: Cryptocurrency is a decentralised digital currency...
```

[Confidence: 1.000 | Intent: crypto]

You: quit

CapitalIQ: It was a pleasure advising you today!

[Session transcript displayed]